

第14讲

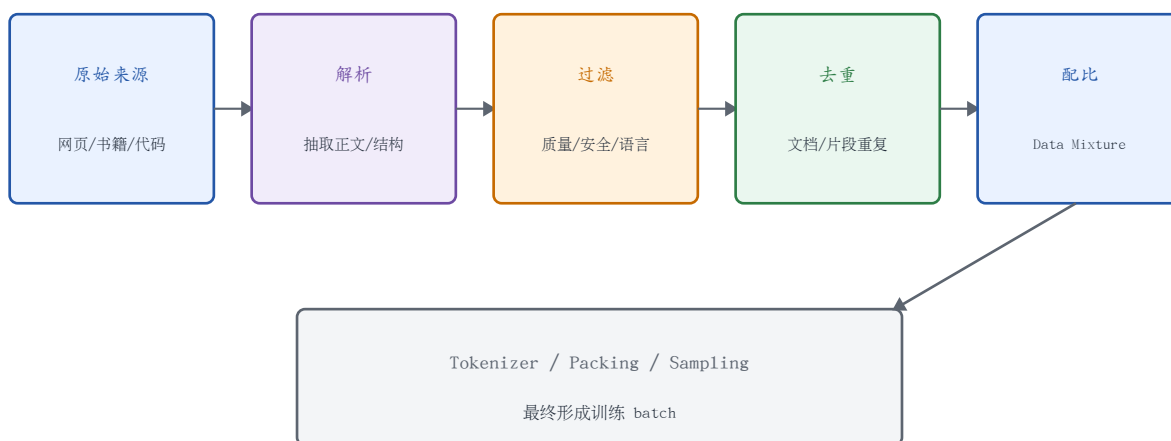
训练数据：大语言模型到底“吃”什么长大？

从数据来源、清洗去重、Data Mixture 到 Synthetic Data 与污染

本讲目标

学完后，你应该能解释：LLM 训练数据从哪里来；为什么原始网页不能直接拿来训练；质量过滤、去重和 Data Mixture 如何改变模型能力；Synthetic Data 为什么既有价值又有风险；Benchmark Contamination、隐私与版权为什么必须在实验中单独考虑。

训练数据不是“下载完就喂给模型”



每一步都会改变“模型最终看到的世界”。

先记住一句

训练数据不是模型的“课本文件夹”，而是决定模型每一步梯度来自哪里的统计环境。数据分布改变，模型学到的参数分布也会改变。

1. 为什么说“数据”是 LLM 的第二个核心组成部分？

前面已经学过：模型架构决定“怎么计算”，优化器决定“怎么更新”，而训练数据决定“模型长期反复看到什么”。如果一个模型几乎从没见过代码，再好的 Transformer 也很难凭空获得强代码能力。

因素	它主要决定什么
架构	信息怎样流动、模型容量与计算形式
训练目标	什么行为会得到更低 loss
训练数据	哪些语言、事实、模式、任务被反复提供
算力	模型能做多少次有效更新

类比

同样的学生、同样的学习方法，如果长期读的教材完全不同，最后形成的知识结构也会不同。数据对模型就是类似的作用。

2. 训练数据一般来自哪里？

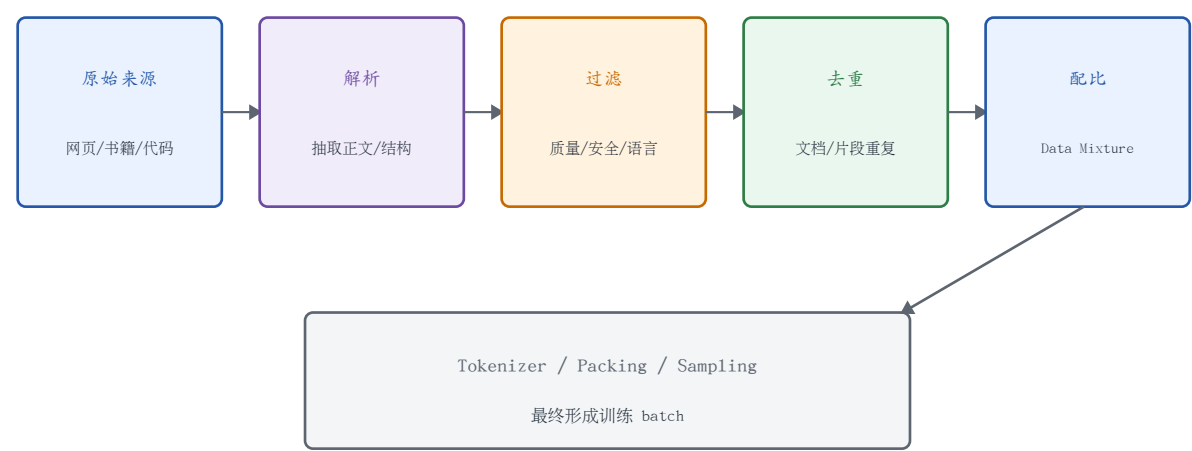
- 大规模网页语料：覆盖广，但噪声、重复和低质量内容也多；
- 书籍与长文：通常结构完整，适合学习长程叙述与知识表达；
- 百科、知识型页面：事实密度较高，但覆盖有限；
- 代码仓库与技术文档：为编程和工具使用提供模式；
- 论文、数学和科学文本：提升专业领域覆盖；
- 论坛、问答、对话等：包含自然交互和问题解决形式；
- 人工构造或合成数据：用于补足稀缺能力与特定任务。

事实边界

商业模型通常不会公开完整数据清单和精确配比。学习时应理解通用类别，不要把某个公开猜测当成确定事实。

3. 原始网页为什么不能直接训练？

训练数据不是“下载完就喂给模型”



每一步都会改变“模型最终看到的世界”。

- 网页里有导航栏、广告、Cookie 提示、脚本、页脚等非正文；
- 同一文章可能被大量转载，导致重复；
- 爬虫可能拿到乱码、空页面、机器生成垃圾页；
- 某些页面语言混杂、结构损坏或质量极低；
- 还可能包含不希望进入训练的数据，例如个人信息或特定受限内容。

数据工程本质

预训练语料构建不是“下载数据”，而是一个复杂的数据产品与数据治理工程。

4. Parsing：先把网页变成“可训练文本”

网页本质上是 HTML/DOM 结构。训练管道需要抽取正文、标题、代码块、列表等真正有意义的内容，同时尽量删除导航、广告和模板。

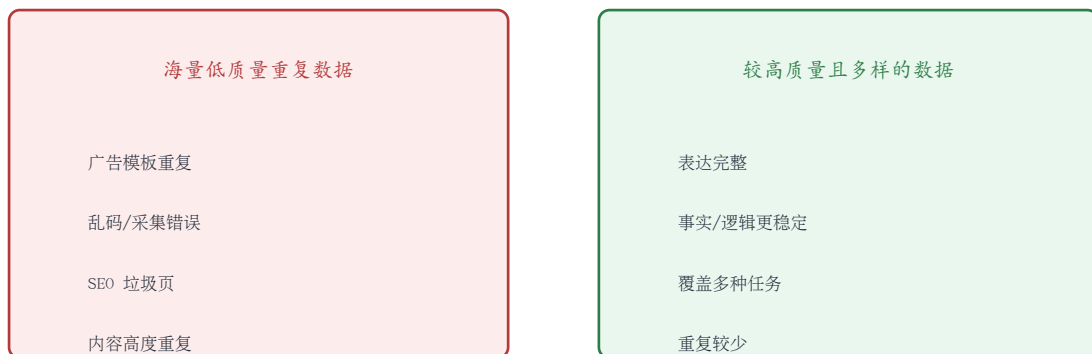
原始网页元素	是否通常保留	原因
正文段落	通常保留	核心语义内容
标题/小标题	常保留	提供结构信息
代码块	视目标保留	对代码模型非常有价值
菜单/导航	通常删除	模板化、重复且信息密度低
广告/推荐位	通常删除	噪声大
脚本/CSS	通常删除	不是自然训练文本

难点

“正文抽取”没有一个完美规则。过度清洗可能删掉有价值结构，清洗不足又会把大量模板噪声带进语料。

5. Quality Filtering: 什么叫“高质量数据”？

数据规模大，不等于训练数据好



训练效果由“规模、质量、多样性、配比、模型容量和算力”共同决定。

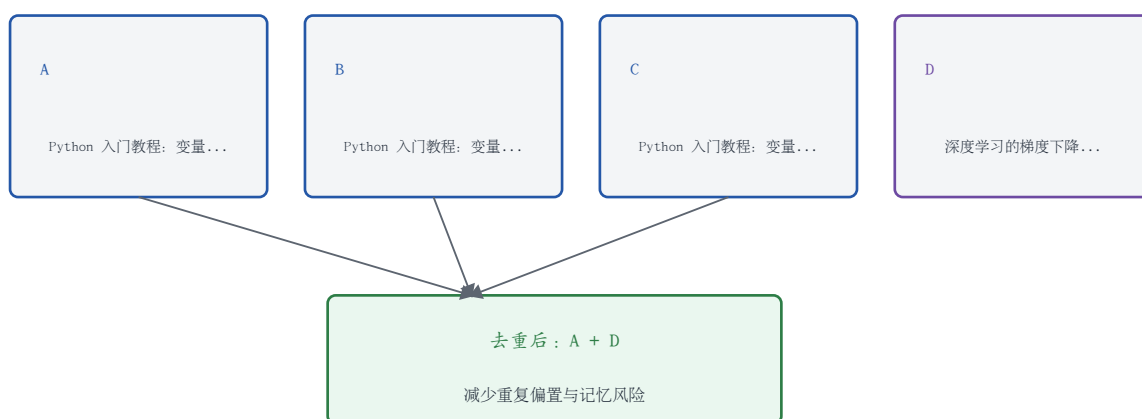
高质量不是一个单一分数。不同训练目标会采用不同标准。例如：语言是否自然、结构是否完整、事实是否较可靠、是否包含有用推理过程、是否为垃圾 SEO 页面、是否高度模板化等。

不要走极端

不是“只保留专家写的内容”就一定最好。过度过滤会损失语言多样性、口语、真实用户表达和长尾知识。真正问题是怎样平衡质量与覆盖。

6. 去重 Dedup: 为什么重复数据会造成问题？

为什么要 Dedup (去重) ?



实际去重可以在文档级、段落级、n-gram 或近似相似度层面进行。

- 重复内容会浪费训练 token;
- 高频重复会人为放大某些来源的权重;
- 更容易强化逐字记忆;
- 公开测试题如果被重复转载, 污染风险会更高;
- 近似重复比完全重复更难处理, 例如轻微改写、转载摘要。

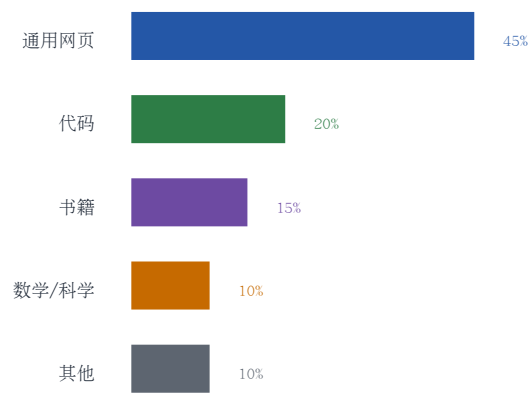
去重层级

可以做 URL 级、文档级、段落级、n-gram

级或向量相似度级去重。越严格越可能删掉真正不同但相似的文本, 因此要权衡。

7. Data Mixture：为什么数据“比例”也非常重要？

Data Mixture：同样 1 万亿 token，配方不同，模型可能很不一样



上图只是教学示例，不代表任何具体模型的真实数据配方。

预训练不是把所有可得数据简单拼起来。不同来源通常会被采样、加权或重采样。增加代码比例，可能提高代码能力；增加数学数据，可能帮助数学模式学习，但也可能牺牲其他分布上的覆盖。

科研视角

Data Mixture
本质上是在控制梯度来源的概率分布。某类数据被采样得越多，它对参数更新的长期影响就越大。

8. “更多数据”为什么不一定一直更好？

更多数据通常有价值，但前提是模型、算力和数据质量匹配。如果只是加入大量重复、低质量或分布极偏的数据，新增 token 的边际价值会很低。

情况	可能结果
大量高质量且互补的新数据	通常有帮助
大量近重复数据	浪费 token，并放大记忆
大量低质量网页	可能稀释高质量信号
极端增加单一领域数据	该领域增强，但整体分布可能偏移
数据远超可用训练算力	很多数据根本没有被充分利用

核心

模型不是按“文件数量”学习，而是按训练过程中实际采样到的 token 和由它们产生的梯度学习。

9. Data Diversity：为什么多样性本身是一种资源？

- 语言多样性：不同语言、方言、表达方式；
- 主题多样性：科技、历史、生活、法律、艺术等；
- 任务多样性：解释、代码、问答、推理、摘要、翻译等；
- 文体多样性：论文、论坛、说明书、对话、故事；
- 难度多样性：基础知识到高难问题。

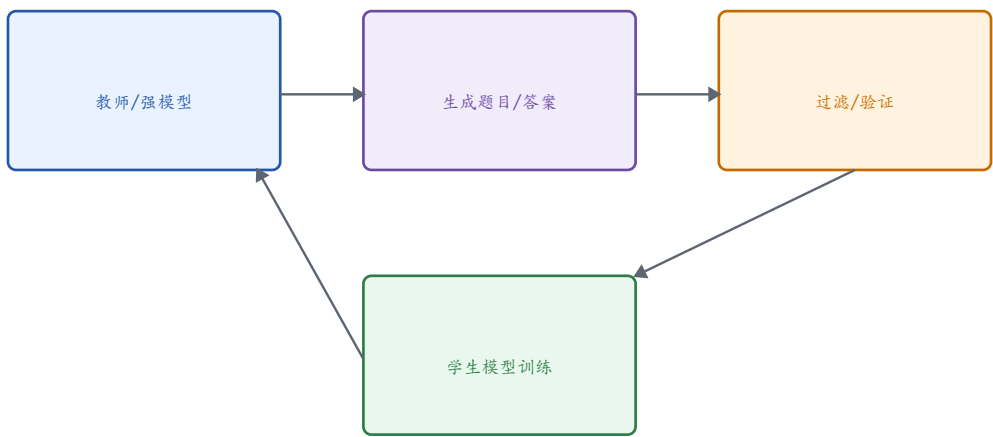
如果训练数据只覆盖狭窄分布，模型在相近任务上可能很强，但遇到新的表达方式或领域时更容易失效。

与第6讲呼应

这和泛化是同一个问题：训练分布决定模型见过哪些变化，测试分布则检验它能否迁移到未见组合。

10. Synthetic Data：为什么越来越重要？

Synthetic Data：模型也可以参与“制造训练数据”



关键不在“是不是合成”，而在生成质量、验证机制、覆盖度和误差是否被放大。

Synthetic Data 指由模型、程序、规则或仿真系统生成的数据。它特别适合扩展稀缺任务，例如数学题、代码测试、工具轨迹、结构化推理过程等。

优点	风险
可低成本扩展特定任务	教师模型错误会被复制
可以控制格式和难度	数据多样性可能不足
可以生成罕见场景	容易产生模板化
可以配合验证器筛选	验证器本身也可能有盲区

关键

Synthetic Data 的价值通常来自“生成 + 验证 + 选择”，而不是单纯让一个模型无限生成文本。

11. Model Collapse / 错误放大：合成数据会不会越训越差？

如果新模型长期只学习前一代模型生成的样本，而且缺少真实数据锚点、质量过滤和多样性控制，错误和偏差可能被重复放大。这个现象常被概括讨论为 model collapse 或 distribution degeneration。

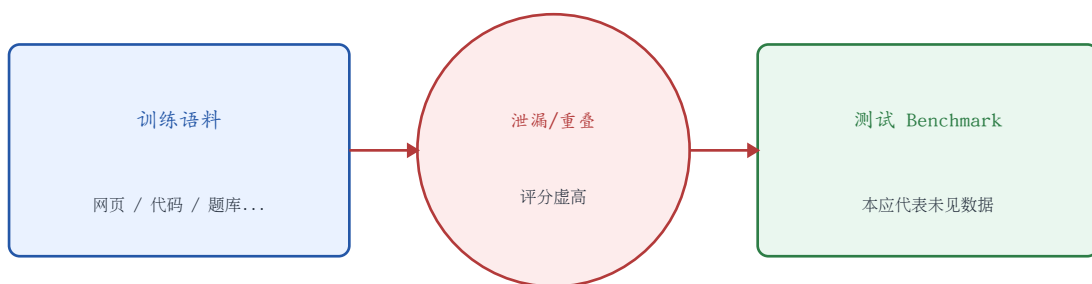
- 错误答案被当成正确标签；
- 少数表达模式被过度复制，分布变窄；
- 罕见但真实的数据逐渐消失；
- 模型越来越擅长模仿自己的输出风格，而不是覆盖真实世界。

不要简单化

这不等于“合成数据一定有害”。高质量合成数据配合可靠验证、真实数据混合和难度控制，可以非常有价值。

12. Benchmark Contamination：为什么“测试题进训练集”很危险？

Benchmark Contamination：测试题如果进过训练集，分数会失真



污染不一定是故意作弊，也可能来自公开 benchmark 被网页转载、题目变体或衍生答案进入预训练语料。

测试集的意义是模拟“模型没有见过的任务”。如果题目、答案、详细解析或近似变体已经进入训练数据，测试分数就可能混入记忆成分，无法纯粹代表泛化能力。

- 精确污染：原题原答案直接出现；
- 近似污染：题目轻微改写但核心内容相同；
- 间接污染：答案解析、论坛讨论或衍生数据泄漏；
- 时间污染：所谓“未来测试集”其实早已在训练截止日前公开。

对论文尤其重要

如果你以后做 Agent Benchmark，必须考虑底座 LLM 可能已经见过 benchmark 本身。否则“Agent 策略更好”和“底座记住题目”可能混在一起。

13. 如何降低 Benchmark Contamination 风险？

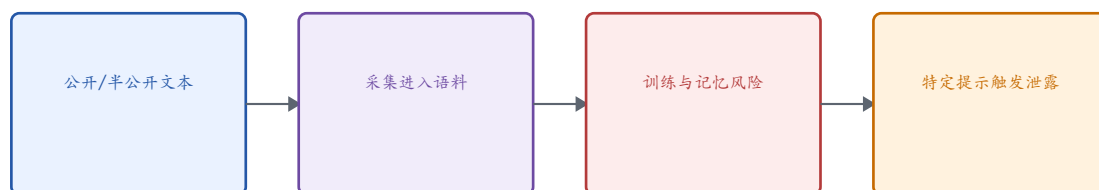
- 使用时间上真正较新的数据；
- 构造私有或新生成的测试集；
- 检查公开搜索结果和重复片段；
- 采用动态 benchmark 或持续刷新题目；
- 设计任务，使成功依赖实时交互而不是单纯回忆答案；
- 报告底座模型、测试集版本和数据截止时间等关键信息。

现实限制

对闭源模型，你通常无法彻底证明“它从未见过某条测试数据”。因此应通过实验设计降低污染可能性，而不是假装可以完全排除。

14. 隐私：训练数据里为什么可能出现敏感信息？

隐私风险的一个基本链条



因此数据治理不仅是“质量工程”，也是隐私、安全和合规问题。

“网页公开可见”不等于“适合无限制进入模型训练”。真实数据可能包含电话号码、邮箱、身份信息、聊天记录、意外暴露的文件等。

- 采集阶段可做敏感信息检测与过滤；
- 去重和频次控制可降低某些内容被高强度记忆；
- 训练后可做记忆与泄露测试；
- 部署阶段还需要输出侧的安全控制。

底层原理

记忆风险通常和样本的独特性、重复频次、模型容量、训练方式以及提示方式有关，不是“模型把所有训练文本逐字存在数据库里”。

15. 版权与许可：为什么它是数据工程中的独立问题？

训练语料可能来自公开网络、开放许可数据、商业授权数据、自有数据或其他来源。是否可抓取、可训练、可再分发并不是同一个问题。具体法律结论取决于地区、许可条款、使用方式和事实背景。

问题	需要区分
能否访问	网页公开可访问
能否复制保存	平台条款与版权许可
能否用于模型训练	法律与许可解释
能否发布原始数据集	再分发权利
模型输出是否复现原文	输出侧风险

科研实践

做公开科研时，优先使用许可清晰的数据集，并记录来源、版本和处理流程。不要把“网上能搜到”自动等同于“可以任意分发”。

16. 数据截止日期 Cutoff 到底是什么？

模型常被描述为有一个知识截止时间，但实际训练流程可能由多阶段数据组成：预训练、继续预训练、指令微调、对齐数据等。不同阶段的数据时间范围未必完全相同。

- Cutoff 是理解“模型知道什么”的粗略线索；
- 它不保证截止日前所有事实都知道；
- 也不保证截止日后的信息绝对不可能出现，因为后续阶段或工具调用可能带来新信息；
- 带 Web / RAG 的系统可以在运行时访问更新数据。

与 Agent Systems 的区别

LLM 参数知识有时间边界，但 Agent 可以通过搜索、数据库和工具把“参数知识”与“实时外部信息”组合起来。

17. Continued

Pretraining：为什么已经训练好的模型还会继续“吃数据”？

基础预训练后，还可以在新的领域数据上继续训练，常称 continued pretraining / domain-adaptive pretraining。目的可能是学习新领域、新语言、新代码库或更新分布。

方式	主要改变什么
继续预训练	通过大量未标注 token 改变基础语言/领域分布
Instruction Tuning	学习“看到指令后怎样回答”
RAG	不改参数，运行时检索外部资料
Fine-tuning	针对特定任务/行为做监督更新

注意遗忘

如果继续预训练过度偏向某个窄领域，模型可能出现能力偏移或遗忘其他分布，因此仍要关注数据混合和训练强度。

18. 数据和能力之间可以一一对应吗？

不能简单一一对应。看到模型会做某项任务，不等于训练集中一定存在同样的题；反过来，训练集中包含某类文本，也不保证模型一定掌握。模型能力来自数据、架构、优化、容量和训练过程共同作用。

错误推理	为什么不严谨
“模型会写代码，所以训练集中一定有这段代码”	可能来自泛化和组合
“训练过数学语料，所以一定会数学推理”	看见数据不等于形成稳健能力
“没见过这道题，所以一定不会”	模型可以组合已有模式解决新题
“测试分数高，所以一定理解了”	可能存在污染、捷径或模板匹配

科研思维

真正需要做的是设计对照实验，区分记忆、模板匹配、统计捷径和可迁移的机制能力。

19. 从数据角度重新理解“幻觉”

幻觉并不只由数据质量造成。即使训练数据全部真实，next-token objective 仍然优化“生成概率”，而不是直接优化“事实真伪”。不过数据中的错误、冲突和过时信息会进一步影响参数分布。

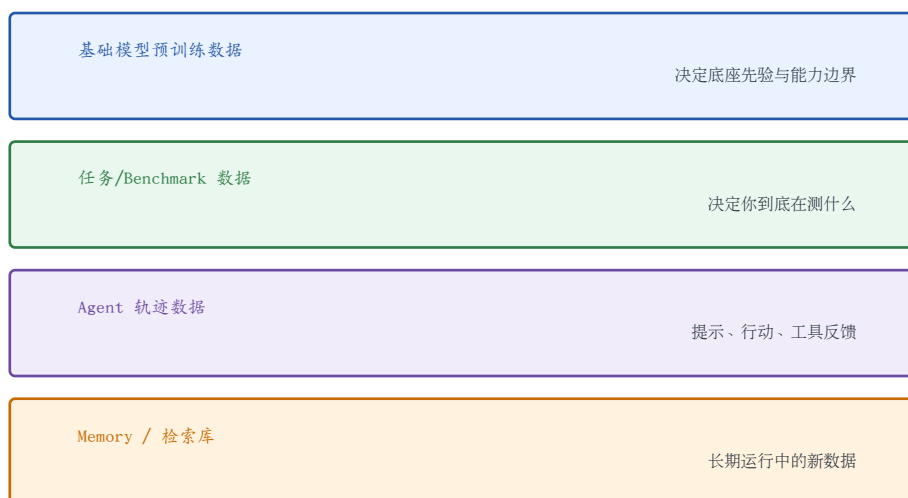
- 同一事实在语料中存在多个互相冲突版本；
- 数据本身过时；
- 长尾事实出现频率极低；
- 模型只学到相关性而没有可验证的事实约束；
- 生成时被上下文诱导到错误分布。

结论

“清洗所有错误数据就能消灭幻觉”是不现实的。事实可靠性还需要检索、验证、工具、推理与系统级控制。

20. 数据问题如何直接进入你的 Agent Systems 科研？

Agent Systems 研究里，数据问题会从多个层级重新出现



研究可靠性时，必须区分：模型本身问题，还是数据分布/污染/采样造成的假象。

- Agent benchmark 是否被底座模型预训练见过；
- Agent 的 memory 是否混入了错误或重复内容；
- 多 Agent 之间是否共享同一数据，从而造成错误高度相关；
- 不同 Agent 的“长期经历”本质上就是不同的数据暴露轨迹；
- 工具返回的数据质量会不会改变协作和决策；
- Synthetic trajectories 能否训练更可靠的 Agent 行为。

科研连接

你以后研究“经历造成异质性”时，Experience 本质上就是一个随时间变化的数据分布。不同 Agent 看到不同数据，可能形成不同内部状态、记忆和错误模式。

21. 一个训练数据审计清单

检查问题	为什么问
数据从哪里来？	判断覆盖、许可与偏差
做过什么清洗？	理解噪声和删减
怎样去重？	判断重复/污染风险
各来源比例多少？	理解能力偏向
是否含合成数据？	评估错误放大与验证方式
是否可能含测试集？	判断 benchmark 有效性
是否有隐私/敏感信息？	判断安全与合规
版本和时间范围是什么？	保证可复现与截止时间解释

最低闭环能力

以后看到一篇 LLM/Agent 论文，不必立刻知道所有数据细节，但至少能问出这 8 个问题。

22. 常见误区

误区	更准确的理解
“数据越多越好”	规模重要，但质量、重复、配比和算力匹配同样重要
“公开网页都能随便训练”	公开可访问与许可/合规不是同一件事
“去重就是删完全一样的文本”	还要处理近似重复和模板化内容
“合成数据都是低质量”	质量取决于生成、验证和采样流程
“测试集污染等于作弊”	也可能是开放网络环境下的无意泄漏
“训练数据就是模型的数据库”	参数是统计压缩后的数值结构，不是可索引原文库

一句话总结

数据不是模型外部的背景条件，而是训练算法持续作用的环境。你改变数据分布，就在改变模型长期接受的梯度分布。

23. 小实验：亲手做一次迷你“数据清洗 + 去重 + 配比 + 污染检查”

材料包里的 Python 脚本不训练真正 LLM，而是模拟训练数据管道的关键步骤。它会：

- 创建多个来源的玩具文档；
- 删除明显垃圾和过短文本；
- 对完全重复和近似重复做检测；
- 统计清洗前后的字符/token 代理数量；
- 按指定 Data Mixture 进行抽样；
- 检查一个“benchmark 题目”是否出现在语料中；
- 输出每一步的数据统计。

运行命令：

```
python 04_training_data_pipeline_demo.py
```

实验目的

不要把重点放在“这个脚本够不够工业级”。重点是建立数据管道意识：训练前的数据选择与处理本身就是实验设计的一部分。

24. 本讲总结：把整条链背下来

原始数据 → 解析 → 质量过滤 → 去重 → 数据配比 → Tokenizer/Packing → 训练 batch → 梯度更新

- 数据来源决定覆盖范围；
- 清洗决定噪声水平；
- 去重决定重复偏置与记忆风险；
- Data Mixture 决定长期梯度来源比例；
- Synthetic Data 可以补足稀缺能力，但必须验证；
- 污染会让 benchmark 失真；
- 隐私、许可与数据治理是独立问题；
- 在 Agent Systems 中，经历、memory、工具结果本质上也都是新的数据流。

进入下一讲

第15讲将进入 Scaling

Law: 为什么参数、数据和算力必须一起扩展，以及为什么“大模型越大越强”不是简单的参数越多越好。